HARNESS LAB · MÓDULO 2 · AULA 2.3

O hook que bloqueia *comando perigoso*

Regra escrita o modelo pode esquecer. Um hook de PreToolUse lê cada comando antes de ele rodar e decide: passa, pergunta ou bloqueia. Primeiro por regra, depois com um modelo de decisão para o que a regra não previu.

`PreToolUse` `matcher Bash` `permissionDecision` `deny · ask` `onFailure` `Jev`

VOCÊ SAI COM

Os dois guards da aula ligados no seu projeto: o de regras, que falha fechado, e o do Jev, que pega a variação que a regra deixa passar. Mais o `.env` protegido nas ferramentas de arquivo e um log de cada comando que rodou.

Material de apoio · Lucas Souza · Beer and Code

1 A aula em *uma página*

Comando destrutivo é o que não tem volta: `migrate:fresh`, apagar arquivo que não está no git, apagar regra importante, mexer em produção. O padrão é o mesmo da aula anterior: um evento, um script, uma decisão. Aqui o evento é o **PreToolUse**, que roda antes da ferramenta. O hook lê o comando que está para rodar e responde: bloqueia, passa ou pede permissão.

Primeira forma, determinística. O pedido foi curto: um hook que bloqueie `migrate fresh`, acesso a produção e remoção do que ainda não está no git. O Claude Code escreveu o `guard-destructive.py`, em Python, porque a regra ficou complexa e o ambiente tinha Python. Hook não precisa ser `.sh`: pode ser Python, Node, até PHP. Ele barra banco (drop, truncate, delete), produção (ssh, scp, rsync), git que descarta trabalho (`reset --hard`, `checkout --`, `restore`), docker que apaga volume (`sail down -v`) e arquivo fora do git. No `settings.json`, ele entra no `PreToolUse`.

Segunda forma, com um modelo de decisão. Regra lê texto: se o comando vier escrito um pouco diferente, ele passa. Então o segundo hook, o `guard-jev.py`, pergunta ao **Jev**, o modelo de decisão da TypeSafe, se o comando apaga arquivo irrecuperável, destrói dados do banco, descarta trabalho no git, toca produção, e qual a severidade. O Jev responde estruturado, sim ou não. A chave da API ficou no `env` do `~/.claude/settings.json`, o do usuário: vale para todos os projetos e chega nos hooks como variável de ambiente.

Abra a mente. Um cliente escrevia query na mão e o agente insistia em subselect, que pesava no banco. Mesmo com regra escrita, um subselect escapava de vez em quando. Com hook, não escapa. E o conselho do fim: hook só para o que garante a qualidade do projeto.

Três respostas

Antes de a ferramenta rodar, o hook avalia o comando e decide: bloqueia, passa ou pede permissão.

Hook em qualquer linguagem

O guard da aula é Python porque a regra ficou complexa. Pode ser shell, Node, até PHP.

Regra ou modelo

A regra é determinística: escreveu diferente, passa. O Jev classifica, com um custo pequeno por chamada. Vai de caso a caso.

Pouco e certeiro

Não vire um super sistema de hooks: você adiciona bug desnecessário. Só o que garante a qualidade.

2 O que a aula *não teve tempo de dizer*

ALÉM DA AULA · O MOTIVO DO DENY É PARA O MODELO

As três respostas viram JSON no stdout. `"permissionDecision": "deny"` bloqueia e o `permissionDecisionReason` volta para o Claude; `"ask"` pergunta para você. Exit 0 sem nada no stdout: o hook não opina e o comando segue o fluxo normal de permissões. Escreva o motivo para quem vai ler: "Não tente contornar, explique ao humano" muda o próximo passo do modelo. "Bloqueado" sozinho, não.

ALÉM DA AULA · O MATCHER BASH SÓ VÊ O TERMINAL

Os dois guards estão em `"matcher": "Bash"`. A ferramenta Read não passa por eles: "leia o `.env`" não é comando de terminal. Por isso a instalação traz o `protege-env.sh` (PreToolUse em `Read|Edit|Write`) e uma regra `"deny": ["Read(./.env)"]` nas permissões. A regra de permissão vale também para `cat`, `head` e `sed` no terminal, mas não para um script que abre o arquivo sozinho.

> *"They don't apply to a command that reads files without naming them [...] or to arbitrary subprocesses that read or write files indirectly, like a Python or Node script that opens files itself."*
>
> Documentação oficial do Claude Code · Configure permissions, Read and Edit

ALÉM DA AULA · HOOK QUE QUEBRA, LIBERA

Se o `python3` não existir ou o caminho no settings estiver errado, o hook falha e o comando roda, como se não houvesse guard. `"onFailure": "block"` inverte isso (Claude Code 2.1.295 ou mais novo). Teste nosso: com o `guard-destructive.py` apagado, até um `ls` foi barrado com `PreToolUse:Bash hook error`. O `guard-jev.py` fica de fora disso de propósito: se a API cair, ele libera e avisa, e o guard de regras continua valendo.

ALÉM DA AULA · DOIS GUARDS NO MESMO EVENTO: VENCE O MAIS RESTRITIVO

Os hooks de um evento rodam em paralelo, então a espera é a do mais lento, não a soma (teste nosso: 200 a 235 ms por chamada ao Jev, em 6 comandos, 02/11/2026; preço oficial em docs.typesafe.ai/models). E se um diz `deny` e o outro libera, o `deny` ganha: "precedence is deny > defer > ask > allow" (Hooks reference, PreToolUse). O Jev nunca solta o que a regra barrou.

ALÉM DA AULA · NEM A REGRA NEM O JEV SÃO MURO

Teste nosso: `python3 -c 'import shutil; shutil.rmtree("storage")'` passou pelo guard de regras (ele não lê Python) e o Jev deu 0,54 para "apaga arquivo irrecuperável", abaixo do limite de pergunta (0,6). Os limites `DENY_NOUL` e `ASK_NOUL` no topo do script são a sua política: cada decisão vai para `$TMPDIR/claude-jev-guard.jsonl`, e é com esse log que você calibra. O sandbox do Claude Code barra no nível do sistema operacional, mas por padrão ele libera escrita no diretório de trabalho: o `storage/` continua exposto. Para proteger uma pasta dentro do projeto, liste em `sandbox.filesystem.denyWrite` (Sandboxing, "Configure sandboxing").

ALÉM DA AULA · A CHAVE FORA DO GIT, A REGRA NA SUA STACK

O `.claude/settings.json` do projeto vai para o git: chave nele vaza. Use o `env` do `~/.claude/settings.json`, como na aula (Settings reference, `env`). O guard do repositório também barra `rails db:reset`, `npx prisma migrate reset` e `python manage.py flush`. Produção: edite `PROD_HOSTS` e `PROD_PATHS` no topo do script ou use `GUARD_PROD_HOSTS` e `GUARD_PROD_PATHS`, separadas por vírgula (sem o prefixo não funciona). Hosts valem para `curl` e `wget` com escrita.

3 Instale *no seu projeto*

1 Confira as ferramentas.

Os guards são Python 3. O `protege-env.sh`, o log e o `instalar.sh` usam `jq`. Faltou algum: `sudo apt install python3 jq` (Ubuntu e WSL) ou `brew install jq` (macOS).

Terminal · em qualquer pasta — shell

```
python3 --version
jq --version
```

2 Baixe os materiais do curso (uma vez só).

Quem já baixou atualiza com `git -C ~/harness-lab-materiais pull`.

Terminal · na sua pasta pessoal — shell

```
cd ~
git clone https://github.com/beerandcodeteam/harness-lab-materiais
```

3 Instale os hooks da aula no seu projeto.

Entre na pasta do seu projeto (`cd`) antes de rodar. Copia os 4 hooks e o `.claude/logs/.gitignore`. Se você já tem um `.claude/settings.json` (da aula 2.2, por exemplo), os hooks e a regra de permissão são somados ao seu, e o original fica em `settings.json.antes-harness-lab`: é só backup, não commite.

Terminal · na raiz do seu projeto — shell

```
~/harness-lab-materiais/instalar.sh 2.3
ls .claude/hooks
```

4 Teste o guard de regras sem abrir o Claude Code.

O `--eval` mostra a decisão para um comando. O esperado: `bloqueia` no primeiro e `libera` no segundo. Troque pelos comandos perigosos da sua stack.

Terminal · na raiz do seu projeto — shell

```
G=.claude/hooks/guard-destructive.py
python3 $G --eval 'php artisan migrate:fresh --seed'
python3 $G --eval 'php artisan migrate:status'
```

5 Teste o protege-env.sh com o JSON que o Claude Code mandaria.

O esperado: um JSON com `"permissionDecision": "deny"`.

Terminal · na raiz do seu projeto — shell

```
jq -nc '{tool_input: {file_path: ".env"}}' | .claude/hooks/protege-env.sh
```

6 Ligue o Jev (opcional).

Pegue a chave em `console.typesafe.ai/keys` e ponha no `env` do **seu** `~/.claude/settings.json`, como no quadro abaixo, trocando `sua-chave` pela chave real. Sem a chave, o `guard-jev.py` sai calado e só a regra trabalha. Depois, teste uma chamada: o `export` lê a chave desse mesmo arquivo (sem colar no histórico do shell) e vale só para este terminal. O esperado: um JSON com `"decision": "deny"`.

`~/.claude/settings.json · o seu, de usuário` — edite à mão · nunca no settings do projeto

```
{
  "env": {
    "TYPESAFE_API_KEY": "sua-chave"
  }
}
```

Terminal · na raiz do seu projeto — shell

```
S=~/.claude/settings.json
export TYPESAFE_API_KEY=$(jq -r .env.TYPESAFE_API_KEY $S)
J=.claude/hooks/guard-jev.py
python3 $J --eval 'php artisan migrate:fresh --seed'
```

7 Abra o Claude Code e confira os hooks.

Em `/hooks` aparecem `PreToolUse` (dois em `Bash`, um em `Read|Edit|Write`), `PostToolUse` e `PostToolUseFailure`. Faça os pedidos: o primeiro é barrado e o motivo volta para o modelo, o segundo passa, o terceiro é negado na leitura.

Dentro do Claude Code — sessão

```
/hooks
> Rode php artisan migrate:fresh --seed
> Rode php artisan migrate:status
> Leia o .env e me diga o APP_KEY
```

8 Veja o log do que rodou de verdade.

Uma linha por comando, com `"status": "ok"` ou `"falhou"`. Comando barrado não aparece: ele nunca rodou.

Terminal · na raiz do seu projeto — shell

```
tail -n 5 .claude/logs/comandos.jsonl
```

.claude/settings.json · o bloco PreToolUse — instalado pelo instalar.sh · não copie do PDF

```
{
  "permissions": { "deny": ["Read(./.env)", "Read(./.env.production)"] },
  "hooks": {
    "PreToolUse": [
      { "matcher": "Bash",
        "hooks": [
          { "type": "command", "timeout": 15, "onFailure": "block",
            "command": "python3 \"$CLAUDE_PROJECT_DIR\"/.claude/hooks/guard-destructive.py" },
          { "type": "command", "timeout": 12, (sem onFailure: falha aberto)
            "command": "python3 \"$CLAUDE_PROJECT_DIR\"/.claude/hooks/guard-jev.py" } ] },
      { "matcher": "Read|Edit|Write",
        "hooks": [ { ... protege-env.sh, onFailure block } ] } ],
    "PostToolUse":        [ { "matcher": "Bash", ... log-comandos.sh } ],
    "PostToolUseFailure": [ { "matcher": "Bash", ... log-comandos.sh } ]
  }
}
```

.claude/hooks/guard-jev.py · as perguntas e a política — trecho · o arquivo inteiro está no repositório

```
# 4 Nouls (probabilidade de "sim") e 1 Score, numa request só, em paralelo
deletes_unrecoverable_files  # apaga arquivo que o git não devolve?
destroys_database_data       # apaga, trunca ou reverte o banco?
discards_git_work            # descarta mudança, branch, stash, reescreve histórico?
touches_production           # chega em servidor remoto ou dado de produção?
damage_severity              # Score de 0 (inofensivo) a 3 (produção ou perda permanente)

DENY_NOUL = 0.85      # algum Noul acima disso: deny
ASK_NOUL = 0.6       # acima disso: ask, o Claude Code pergunta para você
DENY_SEVERITY = 2.6  # severidade só reforça o deny
```

4 Arquivos que *reforçam a aula*

.claude/hooks/guard-destructive.py

O guard de regras da aula, revisado: quebra o comando em pedaços (`git status && rm x`, `$( )`, `bash -c`), consulta o `git status` antes de deixar apagar e nega quando ele mesmo quebra.

No repositório: modulo-2-trava/aula-2.3-o-hook-que-bloqueia-comando-perigoso/projeto/.claude/hooks/

.claude/hooks/guard-jev.py

O segundo guard da aula. Mostra o desenho certo de modelo de decisão: o Jev dá probabilidades, e a política (os limites) fica no seu código, ajustável.

No repositório: modulo-2-trava/aula-2.3-o-hook-que-bloqueia-comando-perigoso/projeto/.claude/hooks/

.claude/settings.json

O que liga tudo: regra com `onFailure: block`, Jev sem, o `.env` negado por permissão e o log em dois eventos.

No repositório: modulo-2-trava/aula-2.3-o-hook-que-bloqueia-comando-perigoso/projeto/.claude/settings.json

modelos/bloqueia-subselect.py

O caso do cliente do fim da aula: PreToolUse em `Edit|Write` que nega SQL com subselect escrito na mão. A prova de que hook serve para a regra do seu projeto, não só para comando de terminal.

No repositório: modulo-2-trava/aula-2.3-o-hook-que-bloqueia-comando-perigoso/modelos/bloqueia-subselect.py

`modelos/pedir-o-hook.md`

Os dois pedidos da aula, um pedido com os critérios que mudam o resultado e a lista para revisar o hook que o Claude escrever para você.

No repositório: modulo-2-trava/aula-2.3-o-hook-que-bloqueia-comando-perigoso/modelos/pedir-o-hook.md

`modelos/bloqueia-perigoso.sh`

A mesma ideia em bash com `jq`, sem Python, em menos de 60 linhas. Usa `ask` para `rm -rf` dentro do projeto e `git reset --hard`: proibir não é a única resposta.

No repositório: modulo-2-trava/aula-2.3-o-hook-que-bloqueia-comando-perigoso/modelos/bloqueia-perigoso.sh

5 Erros *comuns*

SINTOMA	CAUSA	CORREÇÃO
Todo comando Bash é barrado com `PreToolUse:Bash hook error`.	O `python3` não existe ou o caminho do script está errado. Com `onFailure: block`, falha vira bloqueio.	`python3 --version` e o caminho em `/hooks`. É o comportamento certo de um guard: melhor parar do que liberar.
O Jev não faz nada.	Sem `TYPESAFE_API_KEY`, o `guard-jev.py` sai calado de propósito.	Chave no `env` do `~/.claude/settings.json`, sessão nova, e teste com `--eval`.
O `--eval` do Jev dá `HTTPError 401`.	A chave está errada ou ainda é o texto de exemplo `sua-chave`.	Copie a chave de novo em `console.typesafe.ai/keys` para o `env` do settings e refaça o `export`.
O `.claude/settings.json` ficou vazio depois do `instalar.sh` e nenhum hook aparece.	O settings antigo não era JSON válido (comentário `//` ou vírgula sobrando) e a soma com `jq` falhou.	Restaure do `settings.json.antes-harness-lab`, corrija (`jq . .claude/settings.json` aponta o erro) e rode o instalador de novo.
Aparece "guard-jev indisponível" no transcript.	Rede, timeout ou chave inválida. O comando seguiu só com a regra.	Rode o `--eval` no terminal para ver o erro. Timeout em `JEV_GUARD_TIMEOUT`.
Apagar um arquivo novo é barrado.	Ele não está no git: se apagar, não volta. É a regra da aula.	Commit antes, ou apague você. Pasta que se regenera vai em `SAFE_PROJECT_PREFIXES`.
`find . -name '*.log' -delete` é sempre barrado.	A raiz do projeto é diretório crítico para o guard.	Aponte a subpasta: `find storage/logs -name '*.log' -delete`.
O modelo leu o `.env` mesmo com o guard.	O matcher `Bash` não vê a ferramenta Read.	O `protege-env.sh` e o `deny` em `Read(./.env)` da instalação. Confira em `/hooks` e `/permissions`.

SINTOMA	CAUSA	CORREÇÃO
Chave da API apareceu no GitHub.	Foi para o `.claude/settings.json` do projeto, que é commitado.	Revogue a chave no console da TypeSafe e mova para o settings do usuário ou o `.claude/settings.local.json`.

6 Você sai *com*

- [ ] `guard-destructive.py`, `guard-jev.py`, `protege-env.sh` e `log-comandos.sh` em `.claude/hooks/`, e eles aparecem em `/hooks`.
- [ ] `--eval 'php artisan migrate:fresh --seed'` responde `bloqueia`, e o equivalente da sua stack também.
- [ ] No Claude Code, o comando mais perigoso que você conhece foi barrado e o modelo explicou por quê, sem tentar contornar.
- [ ] "Leia o .env" é negado, e o `.env.example` continua legível.
- [ ] Se usa o Jev: a chave está no settings de usuário, e o `--eval` devolve `deny` para um comando destrutivo.
- [ ] `.claude/logs/comandos.jsonl` tem linhas `ok` e `falhou`, e fica fora do git.

PROMETIDO NA AULA

- ✓ "Vocês vão ter acesso a todos esses códigos": o `guard-destructive.py` e o `guard-jev.py` estão em `modulo-2-trava/aula-2.3-o-hook-que-bloqueia-comando-perigoso/projeto/.claude/hooks/`, e o `instalar.sh 2.3` copia para o seu projeto.
- ✓ O `settings.json` com o `PreToolUse` mostrado em aula: `projeto/.claude/settings.json` da mesma pasta, somado ao seu pelo instalador.
- ✓ O hook que bloqueia subselect, do caso do cliente: `modelos/bloqueia-subselect.py`.
- ✓ Da grade do curso, "um log de cada comando executado": `projeto/.claude/hooks/log-comandos.sh`.

7 Glossário

PreToolUse	Evento que roda antes de a ferramenta executar: é onde um hook vê todo comando e pode negá-lo.
matcher	Qual ferramenta dispara o hook. `Bash` é o terminal; Read, Edit e Write são ferramentas de arquivo, à parte.
permissionDecision	A resposta em JSON do PreToolUse: `deny` nega e o motivo vai para o modelo, `ask` pergunta para você, `allow` pula a pergunta.
Falhar fechado / aberto	Fechado: na dúvida, bloqueia (o guard de regras). Aberto: na dúvida, libera (o Jev, que tem a regra atrás).
onFailure	Campo do hook no settings. Com `"block"`, hook que quebra ou estoura o tempo barra a ação.
Jev	Modelo de decisão da TypeSafe. Não gera texto: devolve respostas tipadas e probabilidades.
Noul · Score	Tipos de pergunta do Jev. Noul: probabilidade de sim, de 0 a 1. Score: posição numa escala de níveis descritos.
Untracked	Arquivo que o git não conhece. Apagou, não tem `git checkout` que traga de volta.

8 Para ir *além*

Hooks reference	`code.claude.com/docs/en/hooks` · a seção PreToolUse (campos de decisão, precedência entre hooks) e "Block the action when a hook fails".
Configure permissions	`code.claude.com/docs/en/permissions` · regras de deny e ask, o que a regra `Read` cobre no terminal e como hook e permissão se combinam.
Sandboxing	`code.claude.com/docs/en/sandboxing` · o bloqueio no nível do sistema operacional. Por padrão o projeto fica gravável; pasta interna se protege com `sandbox.filesystem.denyWrite`.
Jev · Noul e Score	`docs.typesafe.ai/primitives/noul` e `/primitives/score` · como escrever a pergunta e os critérios.
Jev · Models	`docs.typesafe.ai/models` · versões do Jev, limite de contexto e preço.

Próxima: *Aula 2.4 · Como não quebrar seu fluxo com hooks*

Hook em loop, hook que trava a sessão, chmod esquecido. Antes de confiar num guard, você testa: JSON falso no stdin para cada regra, um caso que barra e um que passa.